

Amenly

FRONTEND GUIDE

React 18 + Vite single-page app

70 source modules · React Router · Axios · Framer Motion · Recharts · Tailwind CSS

Architecture at a glance

Stack. React 18 with Vite. Routing via React Router; data fetching via Axios; animation via Framer Motion; charts via Recharts; styling via Tailwind CSS. The live AI interview uses a WebSocket.

Folder map.

<code>src/pages</code>	One component per route (screens).
<code>src/components</code>	Reusable UI, grouped by area (compliance, dashboard, chat, ui, landing).
<code>src/context</code>	Global React context providers (auth, dashboard).
<code>src/hooks</code>	Reusable stateful logic (auth, websocket, frameworks).
<code>src/api</code>	Axios modules; one per backend domain.
<code>src/data / utils / constants</code>	Static data, helpers and shared constants.

Auth flow. The AuthContext logs in against `/api/v1/auth/login`, stores the JWT, and exposes the current user. ProtectedRoute guards private routes; admin-only routes additionally require the admin role. Axios attaches the token and handles 401s.

Data flow. Pages call api modules, which return the backend's `{ success, message, data }` envelope; components render data. The assessment chat streams over a WebSocket managed by `useWebSocket`.

Contents

Application Shell 2

Pages (Routes) 8

Assessment / Interview UI 6

Dashboard UI 13

Legacy Chat UI 6

UI Primitives 4

Shared / Landing Components 13

Global State (Context) 2

Hooks 3

API Layer 8

Static Data 1

Utilities 3

Constants 1

Application Shell

Entry point, routing and top-level providers.

App.jsx

App

Root component. Declares the client-side routes and wraps the tree in the auth and dashboard providers.

main.jsx

main.jsx

Vite entry point. Mounts <App/> into the DOM and imports global styles.

Pages (Routes)

Each page maps to a route in App.jsx.

ROUTES

PATH	COMPONENT	ACCESS
/	LandingPage	Public
/login	AuthRedirect	Public
/signup	AuthRedirect	Public
/dashboard	ProtectedRoute	Admin only
/ai-compliance	ProtectedRoute	Protected
/permissions	ProtectedRoute	Protected
/admin	ProtectedRoute	Admin only
/about	AboutPage	Public
*	NotFoundPage	Public

AIComplianceChat.jsx

AIComplianceChat

The member's assessment experience: a stepped flow (welcome → company profile → framework → AI interview → summary) with the live AI interview and a progress panel.

AboutPage.jsx

AboutPage

Static about/company page.

`AdminPanel.jsx`

AdminPanel

Admin console: manage frameworks, join requests, launch/advance/close assessment campaigns, view before/after + regulation scores, and open each member's interview transcript.

`AuthPage.jsx`

AuthPage

Login and sign-up screen. Handles credential entry, validation, network-error messaging, and post-login routing (admins → dashboard, members → AI compliance).

`DashboardPage.jsx`

DashboardPage

Admin-only dashboard: compliance scores, infrastructure map, vulnerabilities and regulation tracking.

`LandingPage.jsx`

LandingPage

Public marketing home page: hero, features, how-it-works and call-to-action sections.

`NotFoundPage.jsx`

NotFoundPage

404 fallback page for unknown routes.

`PermissionsPage.jsx`

PermissionsPage

Admin screen to view and edit per-user role permissions.

Assessment / Interview UI

The AI-driven compliance interview experience.

`ChatBox.jsx`

ChatBox

AI-driven interview chat. The auditor leads over WebSocket; progress and running score come from AI message metadata.

`FrameworkSelector.jsx`

FrameworkSelector

Framework picker step in the assessment flow.

MessageBubble.jsx

MessageBubble

Audit-console chat bubble (auditor vs member).

ProgressPanel.jsx

ProgressPanel

Live side panel during the interview: compliance gauge, risk, controls progress, framework/org info.

StepForm.jsx

StepForm

Company-profile form step in the assessment flow.

WelcomeScreen.jsx

WelcomeScreen

Intro step that starts the assessment flow.

Dashboard UI

Admin dashboard: scores, infrastructure map, vulnerabilities, regulation.

AIInsightBar.jsx

AIInsightBar

—

AssetDetailPanel.jsx

AssetDetailPanel

Drawer showing a selected asset's details and vulnerabilities.

AssetListView.jsx

AssetListView

—

AssetNode.jsx

AssetNode

A single asset node rendered on the infrastructure map.

AssetTooltip.jsx

AssetTooltip

—

ComplianceCharts.jsx

ComplianceCharts

Charts of compliance scores (before/after) using Recharts.

CountryIcon.jsx

CountryIcon

—

DashboardHeader.jsx

DashboardHeader

—

DepartmentIcon.jsx

DepartmentIcon

—

InfrastructureMap.jsx

InfrastructureMap

Interactive network map of the organization's assets and their connections.

RegulationTracker.jsx

RegulationTracker

Shows the region-regulation alignment score and notes.

StatsCards.jsx

StatsCards

Top-level KPI cards on the dashboard.

StatusIcon.jsx

StatusIcon

—

Legacy Chat UI

Reusable chat primitives.

ChatEngine.jsx

ChatEngine

—

ChatMessage.jsx

ChatMessage

—

OptionButtons.jsx

OptionButtons

—

ProgressBar.jsx

ProgressBar

—

SummaryCard.jsx

SummaryCard

—

TypingIndicator.jsx

TypingIndicator

—

UI Primitives

Small presentational building blocks.

Accordion.jsx

Accordion

—

Badge.jsx

Badge

—

FeatureCard.jsx

FeatureCard

—

TestimonialCard.jsx

TestimonialCard

—

Shared / Landing Components

Landing page sections and shared widgets.

Button.jsx

Button

—

CTASection.jsx

CTASection

—

ErrorBoundary.jsx

ErrorBoundary

—

FeaturesSection.jsx

FeaturesSection

—

Footer.jsx

Footer

—

GlassCard.jsx

GlassCard

—

Hero3DScene.jsx

Hero3DScene

—

HeroSection.jsx

HeroSection

—

HowItWorksSection.jsx

HowItWorksSection

—

LoadingScreen.jsx

LoadingScreen

—

LoadingSpinner.jsx

LoadingSpinner

—

Navbar.jsx

Navbar

—

ProtectedRoute.jsx

ProtectedRoute

Route guard. Redirects unauthenticated users to login and non-admins away from admin-only routes.

Global State (Context)

React context providers for cross-cutting state.

`AuthContext.jsx`

AuthContext

Global authentication state: current user, login/register/logout, token storage, and network-error handling.

`DashboardContext.jsx`

DashboardProvider

Shared dashboard data/state (org, assets, scores) for the dashboard components.

Hooks

Reusable stateful logic.

`useAuth.js`

useAuth

Convenience hook to read the AuthContext (current user + auth actions).

`useFrameworks.js`

useFrameworks

Loads and caches the available compliance frameworks.

`useWebSocket.js`

useWebSocket

Manages the assessment chat WebSocket: connect, send message, and callbacks for messages, typing, connection, error and interview completion.

API Layer

Axios modules that call the backend REST API.

assessments.js

createAssessment

Functions: `createAssessment()`, `listAssessments()`, `getAssessment()`, `startAssessmentSession()`, `getAssessmentSession()`, `completeAssessmentSession()`, `saveAssessmentAnswer()`, `sendChatMessage()`, `getChatHistory()`, `launchCampaign()`, `advanceCampaignPhase()`, `closeCampaign()`, `listCampaignSessions()`, `getCampaignOverview()`, `getMemberSessionChat()`

POST /assessments **GET** /assessments **GET** /assessments/{id}

POST /assessments/{assessmentId}/sessions/start **GET** /assessments/sessions/{sessionId}

POST /assessments/sessions/{sessionId}/complete **POST** /assessments/answers

POST /assessments/sessions/{sessionId}/chat **GET** /assessments/sessions/{sessionId}/chat

POST /assessments/{assessmentId}/launch **POST** /assessments/{assessmentId}/advance-phase

POST /assessments/{assessmentId}/close **GET** /assessments/{assessmentId}/sessions

GET /assessments/{assessmentId}/overview

GET /assessments/{assessmentId}/sessions/{sessionId}/chat

assets.js

extractAssets

Functions: `extractAssets()`, `listInfrastructureAssets()`, `getAssetStatistics()`, `scanVulnerabilities()`, `listVulnerabilities()`, `getInfrastructureMap()`

POST /assets/extract **GET** /assets/ **GET** /assets/statistics **POST** /assets/scan-vulnerabilities

GET /assets/vulnerabilities **GET** /assets/map

auth.js

register

Functions: `register()`, `requestJoin()`, `login()`, `logout()`, `refresh()`, `me()`

POST /auth/register **POST** /auth/join-request **POST** /auth/login **POST** /auth/logout

POST /auth/refresh **GET** /auth/me

client.js

setAccessToken

Axios instance + interceptors: injects the Bearer token, handles 401/refresh, and derives the WebSocket base URL.

Functions: `setAccessToken()`, `getAccessToken()`, `clearAccessToken()`, `getRefreshToken()`, `setRefreshToken()`, `clearRefreshToken()`

dashboard.js

getDashboardOverview

Functions: `getDashboardOverview()`, `getComplianceDashboard()`, `getAssetsDashboard()`, `getRisksDashboard()`

GET /dashboard/overview

GET /dashboard/compliance

GET /dashboard/assets

GET /dashboard/risks

frameworks.js

listFrameworks

Functions: `listFrameworks()`, `getFrameworkStats()`, `getFrameworkTypes()`, `getFrameworkCategories()`, `getFrameworkRegions()`, `getFramework()`, `getAvailableFrameworks()`, `associateFrameworks()`, `dissociateFramework()`

GET /frameworks/

GET /frameworks/stats

GET /frameworks/types

GET /frameworks/categories

GET /frameworks/regions

GET /frameworks/\${id}

GET /frameworks/available/all

POST /frameworks/associate

DELETE /frameworks/dissociate/\${id}

organizations.js

getMyOrganization

Functions: `getMyOrganization()`, `updateCompanyProfile()`, `getJoinRequests()`, `approveJoinRequest()`, `rejectJoinRequest()`, `regenerateInviteCode()`

GET /orgs/me

PUT /orgs/profile

GET /orgs/join-requests

POST /orgs/join-requests/\${id}/approve

POST /orgs/join-requests/\${id}/reject

POST /orgs/invite-code/regenerate

rag.js

getRagHealth

Functions: `getRagHealth()`, `searchRag()`, `queryRag()`

GET /rag/health

POST /rag/search

POST /rag/query

Static Data

Client-side constants and flow definitions.

chatFlowData.js

STEPS

Chat flow data — defines all steps, questions, and framework-specific questions. Structured compliance assessment experience.

Utilities

Helper functions.

`constants.js`

STATUS_COLORS

Shared constants and utilities for the application

`permissions.js`

PERMISSIONS

—

`smoothScroll.js`

LenisStyleScroll

Lenis-Style Smooth Scroll Implementation Mimics the buttery smooth scrolling from moneda.com

Constants

Shared constant values.

`assetConstants.js`

ASSET_TYPES

Asset and compliance presentation constants Used across dashboard components for consistent styling and metadata